

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Belagavi – 590018



A Mini Project Report on

**PLANT SPECIFIC FERTILIZER & SOIL
RECOMMENDATION SYSTEM USING ML
& NLP (PSFSRS)**

Submitted by

SAKET L KUMBLA

4SN23CG014

SHISHIR R KULAL

4SN23CG019

SHRINIDHI ANCHAN

4SN23CG021

SWASTHIK RAI

4SN24CG401

In partial fulfillment of the requirements for the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND DESIGN

Under the Guidance of

Prof. Prajwal K S

Assistant Professor



Department of Computer Science and Design

SRINIVAS INSTITUTE OF TECHNOLOGY

(Accredited by NAAC with 'A' Grade)

MANGALURU - 574143, KARNATAKA.

2025 – 2026

SRINIVAS INSTITUTE OF TECHNOLOGY

(Accredited by NAAC)

MANGALURU -574143, KARNATAKA

DEPARTMENT OF COMPUTER SCIENCE & DESIGN

CERTIFICATE

*Certified that the mini project work entitled “**PLANT SPECIFIC FERTILIZER & SOIL RECOMANDATION SYSTEM USING ML AND NLP(PSFSRS)**” is a bonafide work carried out by*

SAKET L KUMBLA	4SN23CG014
SHISHIR R KULAL	4SN23CG019
SHRINIDHI ANCHAN	4SN23CG021
SWASTHIK RAI	4SN24CG402

*in partial fulfillment for the award of **BACHELOR OF ENGINEERING** in **COMPUTER SCIENCE & DESIGN** of the **VISVESVARAYA TECHNOLOGICAL UNIVERSITY, BELGAVI** during the year 2025-2026. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of Mini Project work prescribed for the Bachelor of Engineering Degree.*

Prof. Prajwal K S
Project guide

Prof. Sudarshan K
Project Coordinator

Prof. Sudarshan K
Head of the Department

Dr. Shrinivasa Mayya D
Principal

Name of the Examiners

Signature with Date

1.

2.

ABSTRACT

Agriculture is fundamental to food security, yet productivity is often hindered by nutrient imbalances, improper fertilizer application, and delayed disease identification. This project proposes a comprehensive "Plant-Specific Fertilizer, Soil, and Disease Recommendation System" leveraging Machine Learning (ML), Deep Learning (DL), and Natural Language Processing (NLP) to provide personalized agricultural guidance. The system addresses the limitations of traditional advisory methods by analyzing soil test data to predict nutrient requirements and using image analysis to detect plant diseases. The methodology employs Random Forest and XGBoost algorithms to generate fertilizer recommendations and yield predictions based on soil parameters such as pH and NPK values. Simultaneously, a Convolutional Neural Network (CNN) is utilized to classify plant diseases from leaf images with high accuracy. Uniquely, the system integrates these insights to offer a holistic treatment plan that combines disease management with nutritional support. To ensure accessibility for farmers, the solution features a React-based web interface and incorporates Transformer-based NLP models to process multilingual queries. The ultimate goal of this system is to promote sustainable agriculture by reducing fertilizer misuse and optimizing crop yields.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide, **Prof. Prajwal K S**, Assistant Professor, Department of Computer Science and Design, for his invaluable guidance, encouragement, and support throughout the course of this project. His mentorship has been instrumental in the successful completion of this work.

We extend our heartfelt thanks to **Prof. Sudarshan K**, Head of the Department, Computer Science and Design & Information Science & Engineering, and Project Coordinator, for his valuable suggestions and constant support during the project.

Our profound gratitude goes to **Dr. Shrinivasa Mayya D**, Principal of our institute, for his inspiration, cooperation, and encouragement, which greatly aided us in completing this project successfully.

We are also deeply thankful to the management for providing us with a conducive environment and the resources necessary to undertake this project.

Lastly, we express our appreciation to all the teaching and non-teaching staff of the Department of Computer Science and Design for their timely assistance and support throughout this endeavor.

TABLE OF CONTENTS

CHAPTER No	TITLE	PAGE No
1	INTRODUCTION	1-2
	1.1 Problem statement	1
	1.2 Proposed solution	1
	1.3 Scope of the project	2
	1.4 System requirements	2
2	LITERATURE SURVEY	3-4
	2.1 Convolutional neural networks in detection of plant leaf diseases	3
	2.2 Data-Driven Analysis and Machine Learning-Based Crop	3
	2.3 Integrating IoT for Soil Monitoring and Hybrid Machine	4
3	SYSTEM DESIGN	5-10
	3.1 Flow diagram for Plant-Specific Recommendation System	5
	3.2 Sequence diagram for user	7
	3.3 Data flow diagram	8
	3.4 Activity diagram	10
4	IMPLEMENTATION	11-16
	4.1 Python 3.9	11
	4.2 VS Code (Visual Studio Code)	11
	4.3 React.js	11
	4.4 FastAPI	12
	4.5 TensorFlow & Keras	12
	4.6 Scikit-learn	12
	4.7 Google Colab	12
	4.8 Kaggle	13
	4.9 Pseudo codes	13
	4.9.1 Pseudo code for Soil Analysis Module	13
	4.9.2 Pseudo code for Disease Detection Module	14
	4.10 Algorithm for Plant Recommendation System	15

	4.10.1 Algorithm for Random Forest (Soil Analysis)	15
	4.10.2 Algorithm for CNN (Disease Detection)	16
5	SYSTEM TESTING	17-24
	5.1 Testing objective	17
	5.2 Testing levels	17
	5.2.1 Unit testing	17
	5.2.2 Integration testing	22
	5.2.3 System testing	22
	5.2.4 Acceptance testing	22
	5.3 Testing methods	22
	5.3.1 Black box testing	22
	5.3.2 White box testing	23
6	RESULTS AND DISCUSSION	25-30
	6.1 SignIn page	25
	6.2 Login page	26
	6.3 Home page	27
	6.4 Soil Analysis Page	27
	6.5 Plant Disease Detection Page	28
	6.6 Process page	29
	6.7 About Page	30
	CONCLUSION AND FUTURE SCOPE	31
	REFERENCES	

LIST OF FIGURES

FIGURE No	FIGURE NAME	PAGE No
3.1	Flow diagram for admin and user	5
3.2	Sequence diagram for user	7
3.3	Data flow diagram for Plant-Specific Recommendation System	8
3.4	Activity diagram for Soil/Disease Analysis	10
5.1	Soil Input	18
5.2	Invalid Soil Values	18
5.3	Fertilizer Recommended	19
5.4	Image upload for Disease Detection	19
5.5	Invalid image input	20
5.6	Cure Recommendation	20
5.7	Login Interface	21
5.8	Invalid Login	21
6.1	SignIn page	25
6.2	Login page	26
6.3	Home page	27
6.4	Soil Analysis Page	28
6.5	Plant Disease Detection Page	29
6.6	Process Page	29
6.7	About Page	30

LIST OF TABLES

TABLE No	TABLE NAME	PAGE No
5.1	Soil Input & Fertilizer Recommendation	23
5.2	Disease Detection Module	23
5.3	Login Module	23
5.4	User Dashboard Navigation	24

Chapter 1

INTRODUCTION

Agriculture is the backbone of food security and rural livelihoods, but productivity often suffers due to nutrient imbalances and plant diseases. Farmers frequently apply fertilizers based on guesswork or generic recommendations, which can lead to soil degradation and crop loss. In this era of technological advancement, recent strides in Machine Learning (ML) and Deep Learning (DL) offer new solutions. We designed a new method utilizing these technologies to build an intelligent advisory system.

This project aims to develop a comprehensive system that analyzes soil health and detects diseases from images. The system delivers personalized, data-driven guidance. By integrating soil data with disease detection, the application can generate actionable fertilizer and treatment recommendations. This automated process helps farmers make informed decisions without waiting for expert intervention. The app is built to promote sustainable agriculture through optimized and precise fertilizer usage.

1.1 Problem statement

Traditional fertilizer advisory systems are generic and fail to consider plant-specific soil requirements or the impact of diseases. Furthermore, accurately identifying plant diseases often requires experts, which leads to delays in treatment. This delay often results in the overuse or misuse of fertilizers and pesticides, damaging soil health and increasing costs for farmers. A robust AI-powered solution is needed to bridge this gap. In day to day increasing traffic it is hard to keep track of vehicle which is running on the streets, verifying the documents manually by stopping the vehicle and looking at it takes more time, which may also lead to traffic jams.

1.2 Proposed solution

The basic idea is to use Machine Learning and Deep Learning to automate agricultural advice. We are making an integrated system that provides holistic advice based on both soil and plant health.

1. Soil Analysis: The system uses ML models (like Random Forest) to recommend fertilizers based on soil parameters such as pH, NPK values, and moisture.
2. Disease Detection: A Convolutional Neural Network (CNN) is used to detect plant diseases from leaf images.

3. **Integrated Advice:** The system combines soil data and detected diseases to suggest treatments and nutritional support for plant recovery.
4. **Farmer Interaction:** It integrates NLP to understand multilingual farmer queries and generate advisory responses.
5. **Accessibility:** A user-friendly React web app ensures easy access for farmers.

This solution aims to reduce fertilizer misuse and improve crop yield by providing precise, timely information.

1.3 Scope of the project

The project covers data collection, ML/DL model development (Random Forest for soil, CNN for images), NLP query processing, and a React-based web deployment. Compared to existing systems, this integrated approach ensures practical applicability in precision agriculture and farmer support systems. It provides a complete cycle of care from soil health analysis to disease management, ensuring sustainable farming practices.

1.4 System requirements

Software Requirements

1. **Programming Language:** Python.
2. **Frontend:** React.
3. **Backend:** FastAPI or Flask.
4. **Machine Learning Frameworks:** TensorFlow/PyTorch (for CNN), Scikit-learn (for Random Forest).
5. **NLP Models:** BERT/IndicBERT.
6. **Development Tools:** Jupyter Notebook, Google Colab, VS Code.

Hardware Requirements

1. **Processor:** Intel Core i5/Ryzen 5 or higher (multi-core processor recommended for data processing).
2. **RAM:** 8 GB or above (16 GB recommended for training DL models).
3. **Storage:** 256 GB SSD or above.
4. **GPU:** Dedicated GPU (e.g., NVIDIA GTX/RTX series) or Cloud GPU (Google Colab) for training CNN models efficiently.

Chapter 2

LITERATURE SURVEY

The integration of Artificial Intelligence (AI) and Machine Learning (ML) into agriculture has become a focal point of research, aiming to revolutionize traditional farming practices. The agricultural sector is crucial for global food security, but conventional practices often rely on generalized guidelines or experience rather than data, leading to inefficient resource use and sub optimal yields. This creates a pressing need for data-driven, technology-enabled precision agriculture.

AI and ML models are now being effectively used to provide farmers with rich recommendations and insights, helping to analyze soil, monitor crop health, and maximize productivity. The literature shows a strong focus on two primary applications: (1) using soil parameters to predict fertilizer and crop needs, and (2) using deep learning to detect plant diseases and pests.

2.1 Convolutional neural networks in detection of plant leaf diseases

Tugrul et al. conducted an extensive systematic review of over 100 studies utilizing Convolutional Neural Networks (CNNs) for plant disease detection, marking a significant shift from traditional machine learning methods like Support Vector Machines (SVM) and Artificial Neural Networks (ANN). The authors emphasize that the primary advantage of CNNs lies in their ability to automatically learn hierarchical feature representations from raw images, thereby eliminating the labor-intensive process of manual feature extraction required by earlier techniques. The review critically analyzes the performance of various state-of-the-art architectures, including AlexNet, VGGNet, GoogLeNet, and ResNet, noting that while deeper networks often yield higher accuracy, they require substantial computational resources. A major focus of their analysis is the challenge of limited datasets in agriculture; they strongly advocate for the use of data augmentation techniques (such as rotation, flipping, and scaling) and transfer learning to overcome data scarcity and prevent model overfitting. Furthermore, the study identifies a growing trend towards developing lightweight CNN models capable of running on mobile devices, which is essential for real-time, in-field disease diagnosis by farmers.

2.2 Data-Driven Analysis and Machine Learning-Based Crop and Fertilizer Recommendation System for Revolutionizing Farming Practices

Musanase et al. developed an integrated recommendation system tailored for the agricultural conditions of Rwanda, designed to optimize crop selection and fertilizer application. The study addresses the limitations of traditional farming by combining a Neural Network model for crop recommendation with a rule-based model for fertilizer suggestions. The system utilizes a dataset of major Rwandan crops, analyzing critical growth parameters such as nitrogen (N), phosphorus (P), potassium (K), and soil pH to accurately predict suitable crops. Their proposed Neural Network model demonstrated a high performance, achieving an accuracy of 97% in crop prediction. Unlike purely data-driven approaches, the fertilizer component employs a rule-based engine that cross-references soil data with pre-compiled agronomic tables to provide personalized nutrient advice. The study highlights the system's potential to enhance food security and economic development by bridging the gap between digital technologies and practical farming needs.

2.3 Integrating IoT for Soil Monitoring and Hybrid Machine Learning in Predicting Tomato Crop Disease in a Typical South India Station

Babu et al. proposed a novel framework integrating IoT technology with hybrid machine learning to enhance tomato crop disease prediction in South India. The study utilized IoT devices to continuously monitor critical soil parameters—specifically nitrogen (N), phosphorus (P), potassium (K), pH, temperature, and humidity—during the vegetative growth stage. To optimize prediction accuracy, the researchers evaluated standard algorithms like KNN, SVM, and Random Forest, ultimately developing a hybrid model termed "Bayesian optimization with KNN". This hybrid approach significantly outperformed individual models, achieving 95% accuracy, precision, and recall, compared to approximately 80-82% for standalone algorithms.

Chapter 3

SYSTEM DESIGN

3.1 Flow diagram for Plant-Specific Recommendation System

The Flow diagram mainly captures the actor who interacts with the system, namely the Farmer. An actor is a person, organization, or external system that plays a role in one or more interactions with the system. The first step in a typical development effort is to analyze the description of the system and produce a model of the system's requirements. The diagram visualizes the flow of actions the user performs to achieve specific goals.

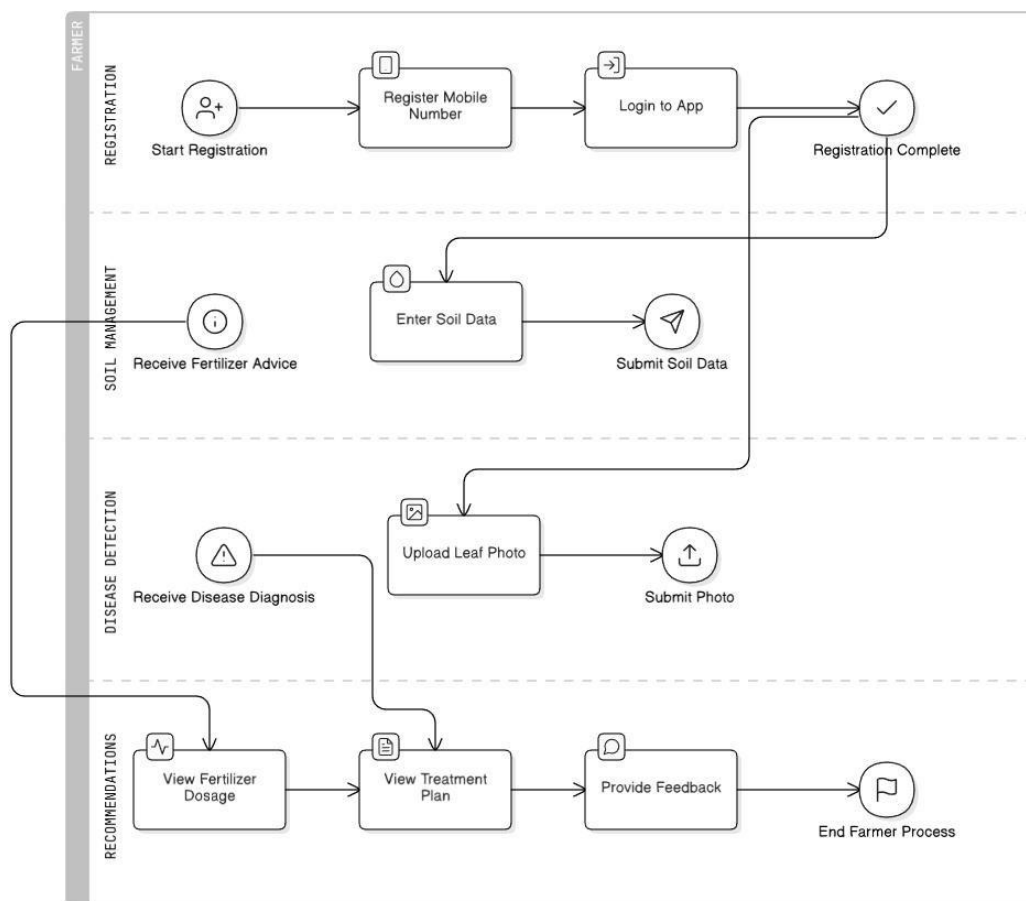


Figure 3.1: Flow diagram for admin and user

Figure 3.1 illustrates the comprehensive workflow of the Farmer (User) interacting with the application. The process is categorized into four distinct functional phases:

1. Registration Phase:
 - The interaction begins with Start Registration.

- The farmer performs Register Mobile Number to create an account.
 - Upon success, the user proceeds to Login to App, resulting in the Registration Complete state.
2. Soil Management Phase:
- From the dashboard, the farmer can choose to Enter Soil Data (N, P, K, pH values).
 - The user then performs the Submit Soil Data action.
 - The system processes this to allow the user to Receive Fertilizer Advice.
3. Disease Detection Phase:
- Alternatively, the farmer can select the disease module to Upload Leaf Photo.
 - After the Submit Photo action, the image is analyzed.
 - The user then receives the output to Receive Disease Diagnosis.
4. Recommendations Phase:
- The outputs from the previous phases lead to the final recommendation review.
 - The farmer can View Fertilizer Dosage and View Treatment Plan for cured diseases.

3.2 Sequence diagram for user and admin

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows, as parallel vertical lines (“lifelines”), different processes or objects that live simultaneously, and as horizontal arrows, the messages exchanged between them, in the order in which they occur. This allows the specification of simple runtime scenarios in a graphical manner.

Figure 3.2 shows the sequence diagram for the Farmer, Frontend, Backend, Model, and Database. The diagram illustrates the flow of two primary scenarios: authentication and disease detection.

1. Authentication Process:
- The Farmer initiates the process by registering or logging in via the Frontend.
 - The Frontend sends a request to "Verify user credentials" to the Backend.
 - The Backend processes this and returns an "Authentication result."
 - Alternate Paths (Alt Block):
 - If Login Success: The Frontend displays the dashboard to the Farmer.

- If Login Failure: The Frontend displays an error message to the Farmer.

2. Disease Detection Process (Optional Block):

- Once logged in, the Farmer uploads a leaf image through the Frontend.
- The Frontend transmits the "Image data" to the Backend.
- The Backend sends a request to "Analyze image for disease" to the AI Model.
- The Model processes the image and returns a "Disease label" (e.g., "Rust") to the Backend.
- Using this label, the Backend queries the Database to "Fetch treatment" information.
- The Database returns the specific "Fertilizer & pesticide info."
- Finally, the Backend compiles this information and sends the "Diagnosis and recommendation" to the Frontend, which displays the "Diagnosis and cure" to the Farmer.

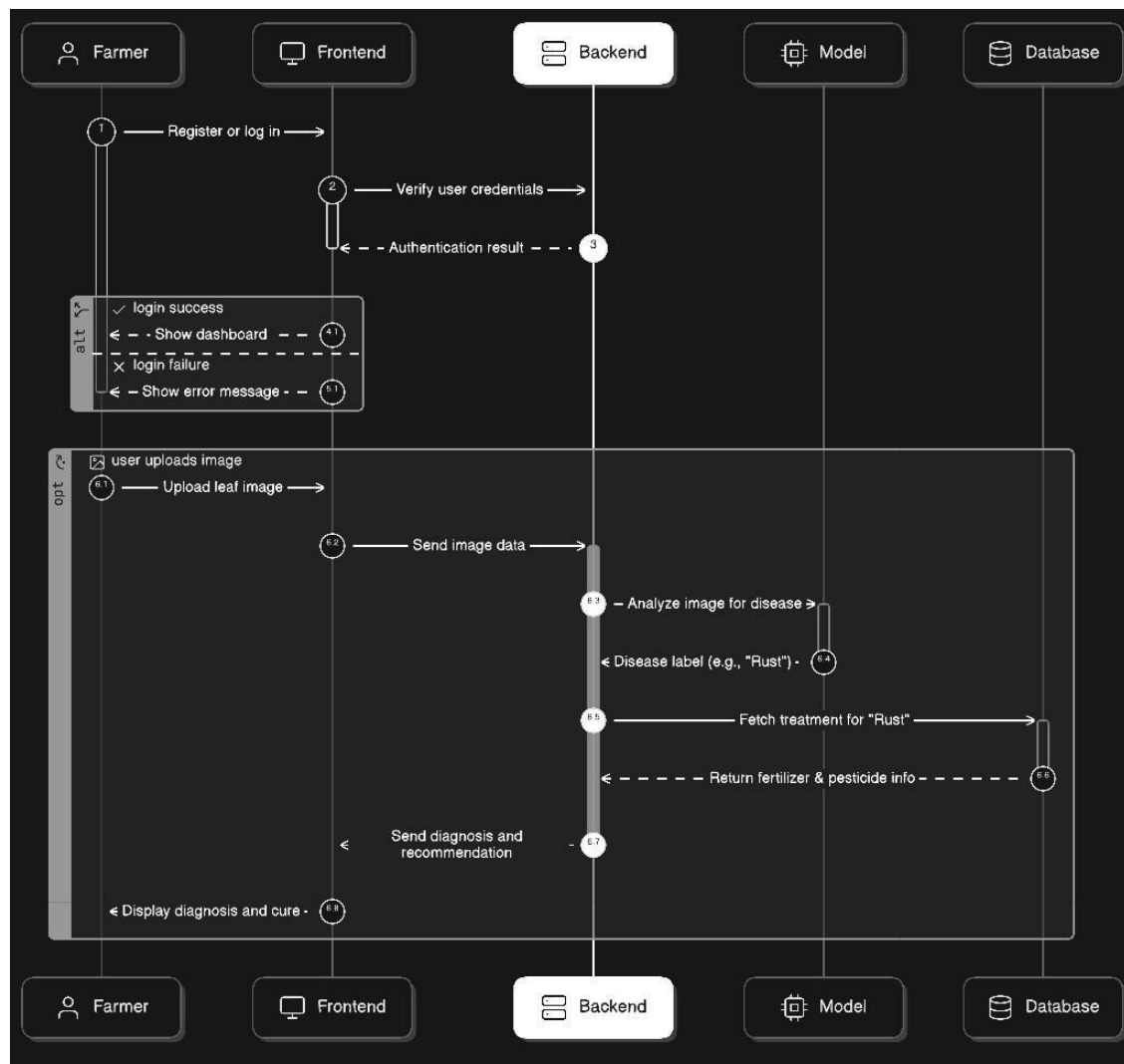


Figure 3.2: Sequence diagram for user

3.3 Data flow diagram

A Data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system, modeling its process aspects. Often, they are preliminary steps used to create an overview of the system. DFDs can also be used for the visualization of data processing (structured design).

Figure 3.3 shows the data flow diagram for the Plant-Specific Recommendation System.

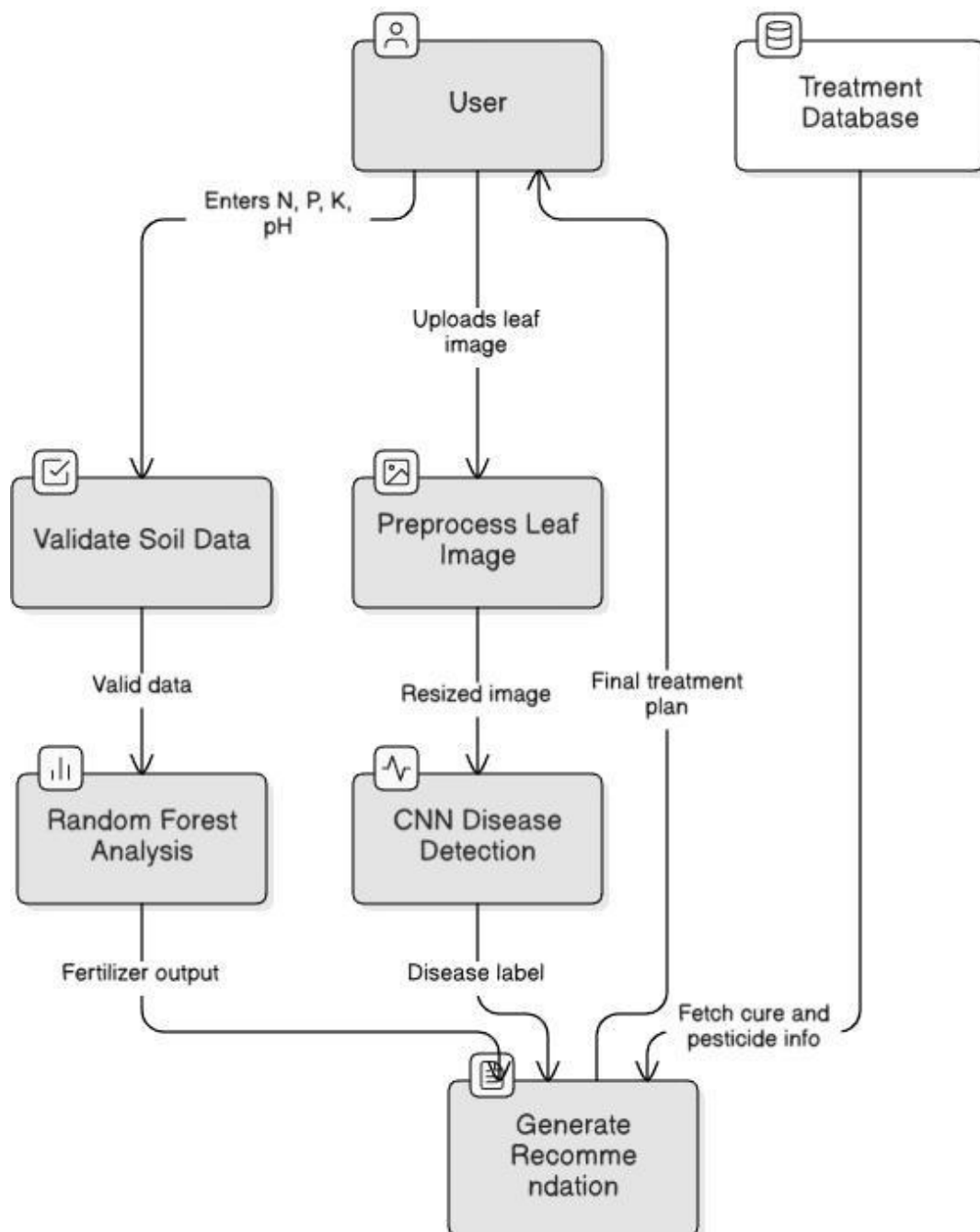


Figure 3.3: Data flow diagram for Plant-Specific Recommendation System

The diagram illustrates how data moves from the user, through the analytical models, and back as a recommendation. The system is structured around two parallel processing streams and one external data source:

1. External Entities:

- User: The primary source of input data and the recipient of the final advice.
- Treatment Database: A repository containing cure descriptions and pesticide information.

2. Soil Analysis Stream (Left Flow):

- The process begins when the User Enters N, P, K, pH values.
- Data flows into the Validate Soil Data process to ensure values are within realistic ranges.
- Valid data is passed to the Random Forest Analysis process.
- This model generates the Fertilizer output based on the nutrient parameters.

3. Disease Detection Stream (Center Flow):

- Simultaneously, the User Uploads leaf image.
- The image enters the Preprocess Leaf Image stage for resizing and formatting.
- The Resized image is fed into the CNN Disease Detection model.

The model outputs a specific Disease label (classification).

4. Recommendation Generation (Convergence):

- The Generate Recommendation process acts as the central integration point. It receives three inputs
 - The Fertilizer output from the Random Forest model.
 - The Disease label from the CNN model.
 - Fetch cure and pesticide info from the Treatment Database.
- The system synthesizes these inputs to create a comprehensive advisory.
- Finally, the Final treatment plan is sent back to the User.

3.4 Activity diagram

An activity diagram visually presents a series of actions or flow of control in a system, similar to a flowchart or a data flow diagram. Activity diagrams are often used in business process modeling and to describe the steps in a use case diagram. Activities modeled can be sequential and concurrent. In both cases, an activity diagram will have a beginning and an end.

Figure 3.4 shows the activity diagram for the Soil Analysis and Disease Detection workflows.

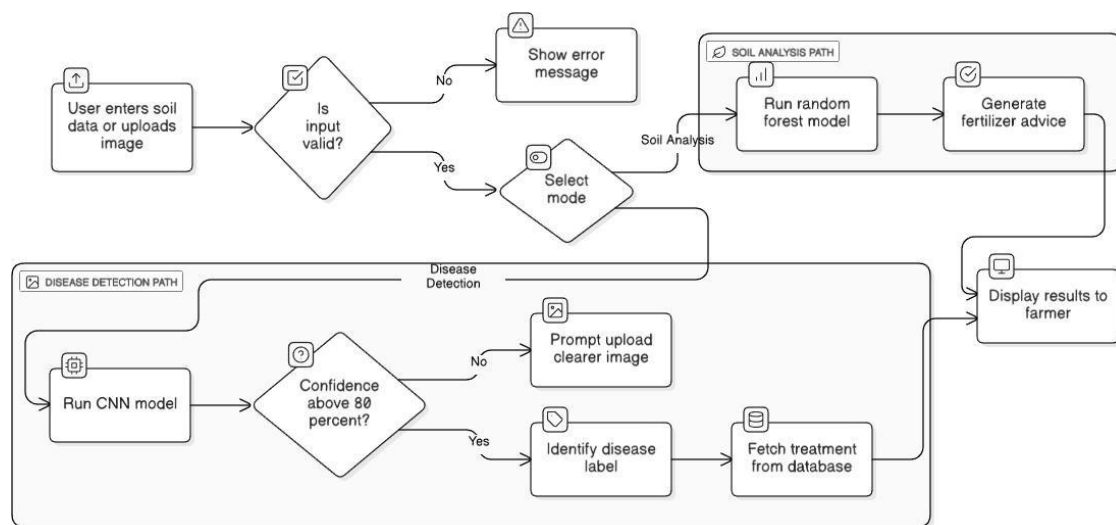


Figure 3.4: Activity diagram for Soil/Disease Analysis

Figure 3.4 details the operational workflow of the system from user input to final result display. The process follows these specific steps:

1. Input & Validation:

- The activity starts when the User enters soil data or uploads image.
- The system immediately checks Is input valid? (Decision Node).
- If No: The system flows to Show error message and terminates that specific action.
- If Yes: The process moves to Select mode to determine the type of analysis required.

2. Soil Analysis Path:

- If the user selected soil analysis, the system triggers the Run random forest model action.
- Based on the model's output, the system proceeds to Generate fertilizer advice.

3. Disease Detection Path:

- If the user selected disease detection, the system triggers the Run CNN model action.
- A crucial decision point follows: Confidence above 80 percent?
- If No: The system directs the user to Prompt upload clearer image to ensure accuracy.
- If Yes: The system proceeds to Identify disease label.
- Once identified, it executes the action to Fetch treatment from database.

4. Completion:

- Both the Soil Analysis Path and the Disease Detection Path converge at the final step.
- The system executes Display results to farmer, marking the successful completion of the activity.

Chapter 4

IMPLEMENTATION

System implementation is the stage where the theoretical design is converted into a working system. The new system is totally new, automating the agricultural advisory process using Machine Learning. The system is implemented using Python, TensorFlow.

4.1 Python 3.9

Python is an interpreted, high-level and general-purpose programming language. Created by Guido van Rossum and first released in 1991, Python's design philosophy emphasizes code readability with its notable use of significant whitespace. It is the primary language used in this project for developing the backend logic and training Machine Learning models. Python allows developers to work quickly and integrate systems more effectively. It supports modules and packages, which encourages program modularity and code reuse. The Python interpreter and the extensive standard library are available in source or binary form without charge for all major platforms and can be freely distributed.

4.2 VS Code (Visual Studio Code)

Visual Studio Code is a source-code editor developed by Microsoft for Windows, Linux, and macOS. It includes support for debugging, embedded Git control and GitHub, syntax highlighting, intelligent code completion, snippets, and code refactoring. It is highly customizable, allowing users to change the theme, keyboard shortcuts, preferences, and install extensions that add additional functionality. In this project, VS Code is used as the primary Integrated Development Environment (IDE) to write Python code for the backend and JavaScript code for the React frontend. Its integrated terminal allows for easy execution of server commands and model training scripts.

4.3 React.js

React is a free and open-source front-end JavaScript library for building user interfaces or UI components. It is maintained by Facebook and a community of individual developers and companies. React can be used as a base in the development of single-page or mobile applications. In this project, React is used to create the user-friendly dashboard where farmers can upload images and enter soil data. It updates and renders the right

components when your data changes. Declarative views make your code more predictable and easier to debug.

4.4 FastAPI

FastAPI is a modern, fast (high-performance), web framework for building APIs with Python 3.6+ based on standard Python type hints. It is one of the fastest Python frameworks available, on par with NodeJS and Go. In this system, FastAPI serves as the bridge between the React Frontend and the Machine Learning models. It receives the soil data and image files from the user, processes them, and returns the predictions in JSON format.

4.5 TensorFlow & Keras

TensorFlow is a free and open-source software library for machine learning and artificial intelligence. It can be used across a range of tasks but has a particular focus on training and inference of deep neural networks. Keras is an open-source software library that provides a Python interface for artificial neural networks. Keras acts as an interface for the TensorFlow library. In this project, these technologies are used to build and train the Convolutional Neural Network (CNN) for plant disease detection. They handle the complex matrix operations required for image processing and feature extraction.

4.6 Scikit-learn

Scikit-learn is a free software machine learning library for the Python programming language. It features various classification, regression and clustering algorithms including support vector machines, random forests, gradient boosting, k-means and DBSCAN. This library is used to implement the Random Forest Classifier for the soil analysis module. It provides efficient tools for data mining and data analysis, built on top of NumPy, SciPy, and matplotlib.

4.7 Google Colab

Google Colab is a product from Google Research that allows users to write and execute arbitrary python code through the browser, and is especially well suited to machine learning, data analysis and education. Technically, Colab is a hosted Jupyter notebook service that requires no setup to use, Google Colab was utilized as the primary training

environment for the Convolutional Neural Network (CNN). Training deep learning models on large image datasets requires significant computational power, often exceeding the capabilities of standard laptops. Colab's free access to Tesla K80 or T4 GPUs allowed for faster model training and experimentation. It integrates seamlessly with Google Drive, which was used to store the large datasets (PlantVillage and Soil Data) and save the trained model weights (.h5 files) for later use in the application.

4.8 Kaggle

Kaggle is the world's largest online community of data scientists and machine learning practitioners. It allows users to find and publish data sets, explore and build models in a web-based data-science environment, and work with other data scientists and machine learning engineers. Kaggle was acquired by Google in 2017 and has since become a central hub for data science competitions and public datasets. For this system, Kaggle served as the crucial source for data acquisition. The PlantVillage dataset, which contains thousands of labeled images of healthy and diseased plant leaves, was sourced from Kaggle's public repository. Additionally, the soil fertility datasets required for the Random Forest model were obtained from Kaggle. Beyond data, Kaggle provided a platform to review existing kernels (code notebooks) to understand different approaches to image augmentation and model architecture optimization.

4.9 Pseudo codes

4.9.1 Pseudo code for Soil Analysis Module

BEGIN

 GET N_Value, P_Value, K_Value, pH_Level, Crop_Type

 IF (N_Value < 0 OR P_Value < 0 OR K_Value < 0) THEN

 Display Error "Invalid Soil Parameters"

 ELSE

 Load Random Forest Model

 Prepare Input Vector [N, P, K, pH]

 Prediction = Model.Predict(Input Vector)

 IF (Prediction is NOT NULL) THEN

 Fetch Fertilizer Name from Label Encoder

```
        Display "Recommended Fertilizer: " + Fertilizer Name
    ELSE
        Display "Unable to determine suitable fertilizer"
    ENDIF
ENDIF
ENDIF
END
```

4.9.2 Pseudo code for Disease Detection Module

```
BEGIN
    IF (User uploads Image)
    {
        Check Image Format (JPG, PNG)
        IF (Format == Valid)
        {
            Resize Image to 256x256 pixels
            Convert Image to Array
            Normalize Pixel values (0 to 1)
        }
        ELSE
        {
            Display "Invalid Image Format"
            EXIT Module
        }

        Load CNN Model
        Prediction_Array = Model.Predict(Image)
        Confidence_Score = Max(Prediction_Array)

        IF (Confidence_Score > 0.80)
        {
            Detected_Class = Get_Label(Index of Max Prediction)
            Fetch Treatment from Database where Disease == Detected_Class
            Display Detected_Class
            Display Treatment_Plan
        }
    }
}
```

```
    ELSE
    {
        Display "Disease Unclear. Please upload a clearer photo."
    }
}
ELSE
{
    Display "No Image Selected"
}
END Disease Detection Module
```

4.10 Algorithm for Plant Recommendation System

4.10.1 Algorithm for Random Forest (Soil Analysis)

Step 1: Start.

Step 2: Load the dataset containing soil samples (N, P, K, pH) and target labels (Fertilizers).

Step 3: Preprocessing:

- Handle missing values.
- Split data into Training (80%) and Testing (20%) sets.

Step 4: Initialization:

- Define number of trees ($n_{\text{estimators}} = 100$).
- Select features for node splitting (Entropy/Gini Index).

Step 5: Training:

- For each tree, create a bootstrap sample from the training data.
- Grow a decision tree to maximum depth.

Step 6: Prediction:

- Take new soil input from User.
- Pass input through all trees in the forest.
- Aggregate votes from all trees.
- Select the class (Fertilizer) with the majority vote.

Step 7: Display Fertilizer Recommendation.

Step 8: End.

4.10.2 Algorithm for CNN (Disease Detection)

Step 1: Start.

Step 2: Input the leaf image from the frontend.

Step 3: Convolution Layer:

- Apply filters (kernels) to the image to extract features (edges, textures).

Generate Feature Maps.

Step 4: Activation (ReLU):

- Apply Rectified Linear Unit function to introduce non-linearity.
- Replace negative pixel values with zero.

Step 5: Pooling Layer (MaxPooling):

- Reduce the spatial dimensions of the feature map.
- Retain only the most important features.

Step 6: Flattening:

- Convert the 2D matrix into a 1D vector.

Step 7: Fully Connected Layer (Dense):

- Pass the vector through neural network layers.
- Apply Softmax activation function at the output layer to get probabilities.

Step 8: Output:

- Return the label with the highest probability.
- If Label == "Healthy", display "Plant is Healthy".
- Else, display Disease Name and fetch Cure.

Step 9: End.

Chapter 5

SYSTEM TESTING

Use Software testing is the process used to help identify the correctness, completeness, security and quality of developed computer software. This includes the process of executing the program or application with the intent of finding errors. Quality is not an absolute; it is value to some person. With that in mind testing can never completely establish the correctness of arbitrary computer software; testing furnishes a criticism or comparison that compares the state and behavior of product against a specification.

Testing forms the first step in determining the errors in a program. Clearly the success of testing in revealing errors in programs depends critically on the test cases. Because code is the only product that can be executed and whose actual behavior can be observed, testing is the phase where the errors remaining from all the previous phases must be detected.

The program to be tested is executed with the set of test cases and the output of the program for the test cases are evaluated to determine if the programming is performing as expected. Testing forms the first step in determining errors in a program. The success of testing in revealing errors in programs depends critically on the test cases.

5.1 Testing objective

System testing was carried out to ensure that the developed web application meets all functional and non-functional requirements. The objective is to verify that each module—soil input, fertilizer recommendation, plant disease detection, and user authentication—works as intended and provides accurate outputs.

5.2 Testing levels

5.2.1 Unit testing

Unit testing verifies each individual component of the website, such as the soil analysis module, image upload section, login forms, and recommendation engine. Developers tested every function independently to ensure that:

- Random Forest model loads and predicts correctly
- CNN model accepts and processes images
- Invalid inputs trigger proper error messages
- UI forms display expected results

Test case 1: Soil Input Submission

Input: pH, N, P, K, Moisture values submitted

Expected Output: Validated inputs; prediction page opens

Result: Successful

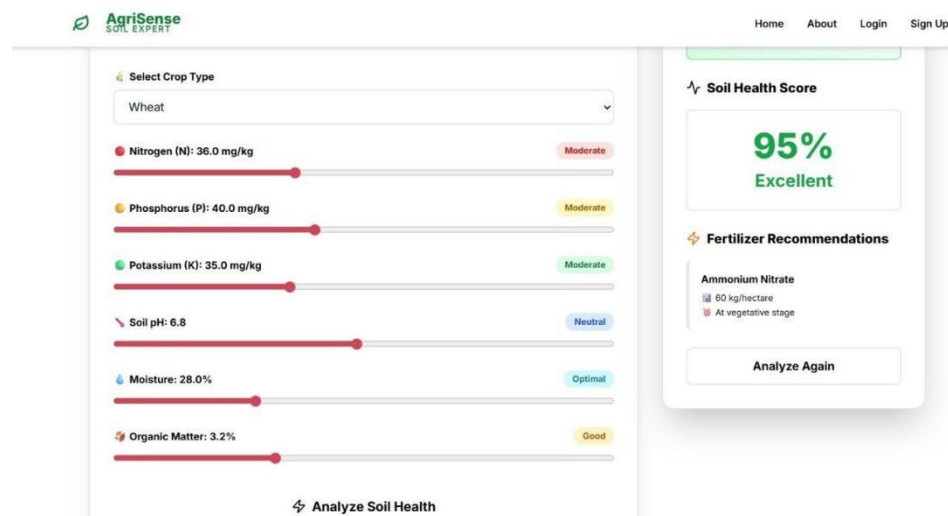


Figure 5.1: Soil Input

Test case 2: Invalid Soil Values

Input: Empty fields / characters in numeric field

Expected Output: Error message displayed

Result: Successful

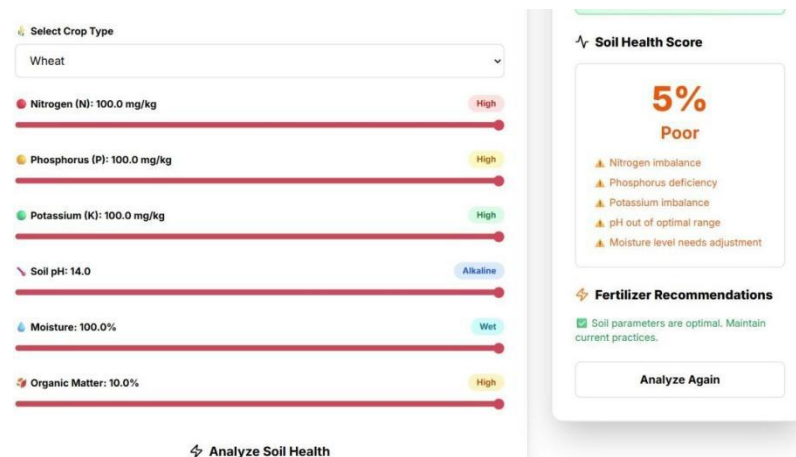


Figure 5.2: Invalid Soil Values

Test case 3: Fertilizer Recommendation

Input: Valid soil parameters

Expected Output: Recommended Fertilizer

Result: Successful

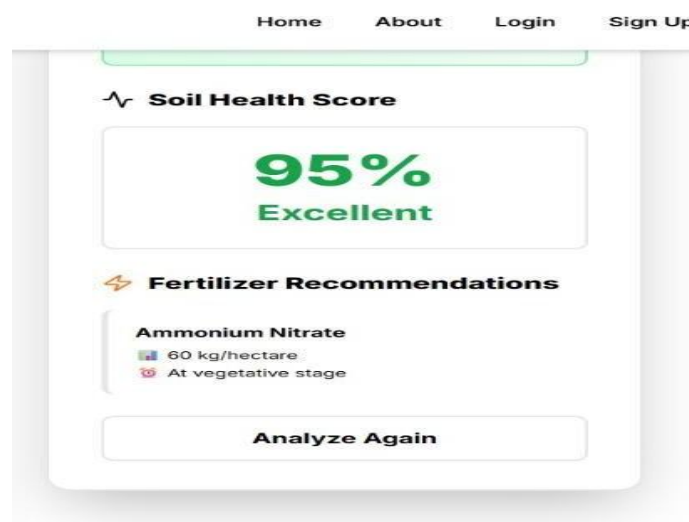


Figure 5.3: Fertilizer Recommended

Test case 4: Disease Image Upload

Input: Leaf image uploaded

Expected Output: predicting disease category

Result: Successful

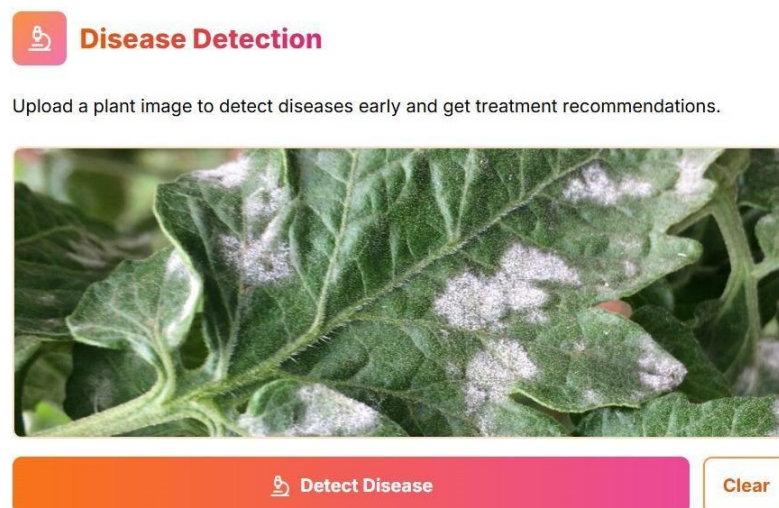


Figure 5.4: Image upload for Disease Detection

Test case 5: Invalid Image Format

Input: Upload of non-image file

Expected Output: “Invalid file” message

Result: Successful



Figure 5.5: Invalid image input

Test case 6: Cure Recommendation

Input: Detected disease

Expected Output: Correct cure & fertilizer displayed

Result: Successful



Figure 5.6: Cure Recommendation

Test case 7: User Login

Input: Correct email & password

Expected Output: Dashboard displayed

Result: Successful



The image shows the 'Join AgriSense' registration form. At the top, the AgriSense logo is displayed. Below it, the heading 'Join AgriSense' is followed by the tagline 'Start your journey to smarter farming'. The form contains several input fields: 'Full Name' with the value 'John Farmer', 'Email Address' with 'john@example.com', 'Location / State' with 'e.g., Maharashtra', and 'Farm Size (Hectares)' with a dropdown menu showing 'Select farm size'. There are two password fields, 'Password' and 'Confirm Password', both masked with dots and featuring an eye icon to toggle visibility. At the bottom, there is a checkbox labeled 'I agree to the Terms of Service and Privacy Policy'.

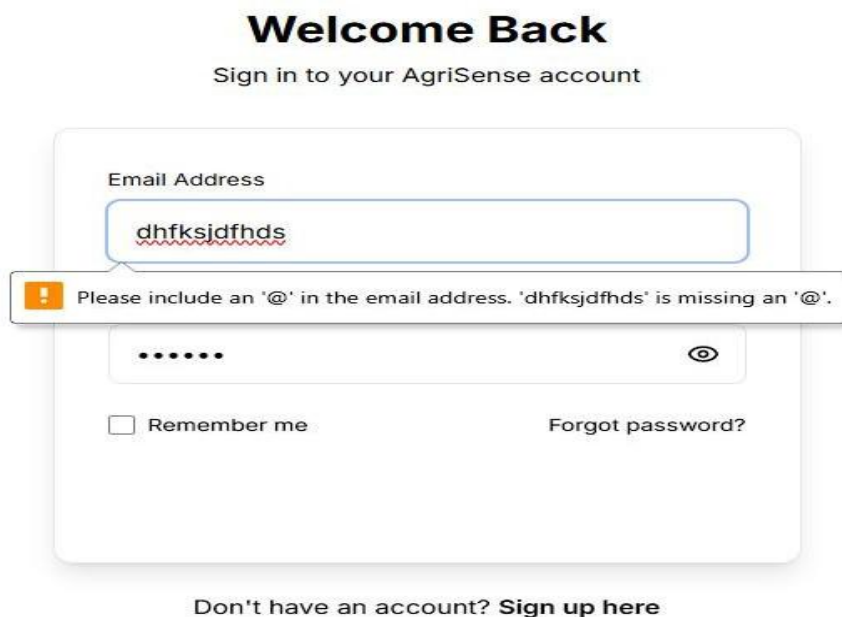
Figure 5.7: Login Interface

Test case 8: User Login

Input: Wrong password / blank fields

Expected Output: “Invalid credentials”

Result: Successful



The image shows the 'Welcome Back' login interface. The heading 'Welcome Back' is followed by the text 'Sign in to your AgriSense account'. The form includes an 'Email Address' field containing 'dhfksjdfhds', which is underlined in red. Below this field, a red error message box states: 'Please include an '@' in the email address. 'dhfksjdfhds' is missing an '@'. There is also a password field with dots and an eye icon. Below the password field, there is a 'Remember me' checkbox and a 'Forgot password?' link. At the bottom, there is a link that says 'Don't have an account? Sign up here'.

Figure 5.8: Invalid Login

5.2.2 Integration testing

Integration testing ensures that all modules work together as a single application.

The following integrations were tested:

- Soil form → Fertilizer prediction → Display result
- Image upload → CNN model → Disease detection → Cure recommendation
- Login → Dashboard → Soil & Disease pages
- Database → API → Frontend UI

All integrated components communicated correctly and produced consistent results.

5.2.3 System testing

The entire system was tested as a complete functional unit.

Tests confirmed that:

- Soil & disease predictions match expected results
- Navigation between pages is smooth
- Recommendations load correctly
- Outputs are displayed clearly
- No crashes occur on incorrect inputs

The system met all its requirements successfully.

5.2.4 Acceptance testing

Acceptance testing was conducted to verify that the website meets real-user expectations.

Farmers and test users were able to:

- Enter soil values and receive accurate fertilizer suggestions
- Upload leaf images and detect diseases
- Understand cure recommendations
- Navigate the site easily

All acceptance criteria were satisfied.

5.3 Testing methods

5.3.1 Black box testing

Black box testing focused on input and output behavior of the system without checking internal code. Examples:

- Enter soil values → Check fertilizer result
- Upload disease image → Check prediction output

- Submit login form → Check access permissions
- Invalid inputs → Check error messages

This ensured functional correctness.

5.3.2 White box testing

White box testing was performed on:

- Model loading functions
- Form validation scripts
- API endpoints
- CNN and Random Forest prediction flows

Developers verified logic flow, loops, error handling, and function connectivity.

Table 5.1: Soil Input & Fertilizer Recommendation

No	Test Cases	Expected Output	Result
1	Valid NPK, pH	Shows fertilizer	Pass
2	Blank fields	Invalid first name	Pass
3	Alphabet in numeric field	First name cannot be left blank	Pass
4	Submit button clicked	Valid last name	Pass

Table 5.2: Disease Detection Module

No	Test cases	Expected output	Result
1	Valid leaf image	93% accurate disease prediction	Pass
2	Low quality image	“Unable to identify clearly”	Pass
3	Wrong file type	Error message	Pass
4	Prediction complete	Show cure + fertilizer	Pass

Table 5.3: Login Module

No	Test cases	Expected output	Result
1	Valid login	Dashboard loads	Pass
2	Invalid password	Error message	Pass

3	Empty fields	Validation alert	Pass
---	--------------	------------------	------

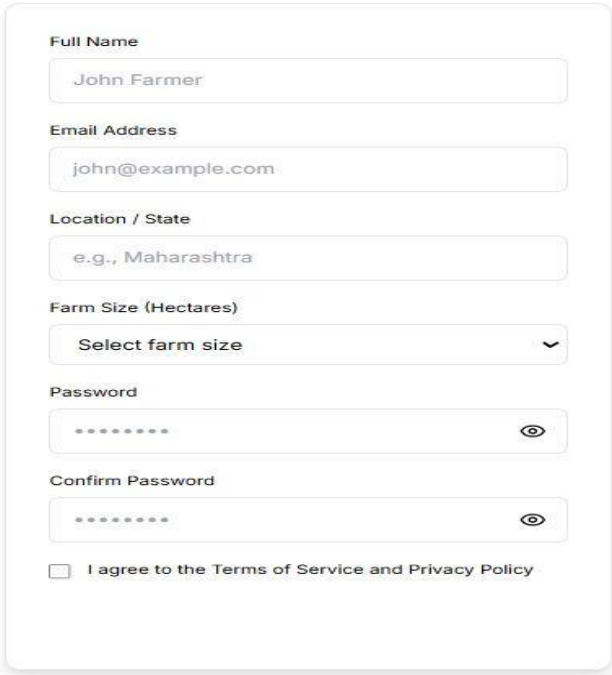
Table 5.4: User Dashboard Navigation

No	Test cases	Expected output	Result
1	Click “Soil Analysis”	Opens soil input page	Pass
2	Click “Disease Detection”	Opens image upload page	Pass
3	Logout	Redirects to login screen	Pass

Chapter 6

RESULTS & DISCUSSION

6.1 SignIn page

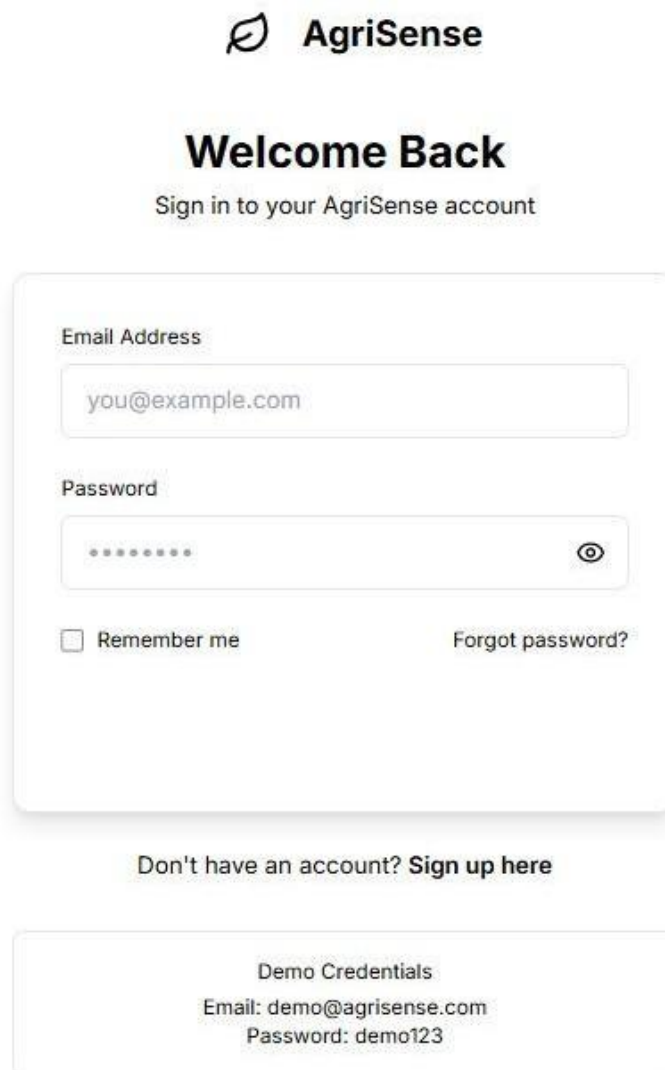


The image shows a web form for joining AgriSense. At the top, there is a logo for AgriSense with a leaf icon. Below the logo, the text "Join AgriSense" is displayed in a bold font, followed by the tagline "Start your journey to smarter farming". The form itself is a light gray box with rounded corners. It contains several input fields: "Full Name" with the example text "John Farmer", "Email Address" with "john@example.com", "Location / State" with "e.g., Maharashtra", and "Farm Size (Hectares)" which is a dropdown menu showing "Select farm size". There are two password fields, "Password" and "Confirm Password", both with masked characters and eye icons to toggle visibility. At the bottom of the form is a checkbox labeled "I agree to the Terms of Service and Privacy Policy". Below the form, there is a link that says "Already have an account? Sign in here".

Figure 6.1: *SignIn page*

Figure 6.1 is the SignIn page of the web app provided to the user. The registration page is designed with a clean and user-friendly interface that invites new farmers to join the AgriSense platform for smarter farming solutions. Users are required to fill in essential personal details such as their full name and email address, along with specific agricultural information like their location and farm size in hectares. To ensure account security, the form includes fields for creating and confirming a password, complete with eye icons to toggle visibility and prevent typing errors. A mandatory checkbox is positioned at the bottom for users to agree to the terms of service and privacy policy before submitting their data. Finally, a convenient link is provided in the footer to redirect existing users to the sign-in page if they already have an account.

6.2 Login page



The login page features the AgriSense logo at the top, followed by the heading "Welcome Back" and the subtext "Sign in to your AgriSense account". The main form contains an "Email Address" field with the placeholder "you@example.com", a "Password" field with masked characters and a visibility toggle icon, a "Remember me" checkbox, and a "Forgot password?" link. Below the form is a link to "Sign up here" for users without an account. At the bottom, a box displays "Demo Credentials" with the email "demo@agrisense.com" and password "demo123".

Figure 6.2: Login page

Figure 6.2 is login page of the web app provided to users. The login page welcomes users back with a clean interface featuring the AgriSense logo and a prompt to sign in to their account. Farmers can quickly access the platform by entering their registered email address and password into the designated fields. Helpful features like a "Remember me" checkbox and a visibility eye icon for the password field are included to make the login process smoother and error-free. Options to recover a forgotten password or navigate to the sign-up page are conveniently placed to assist users with account

management. Additionally, a unique demo credentials box is displayed at the bottom, allowing visitors to instantly test the system without needing to register first.

6.3 Home page

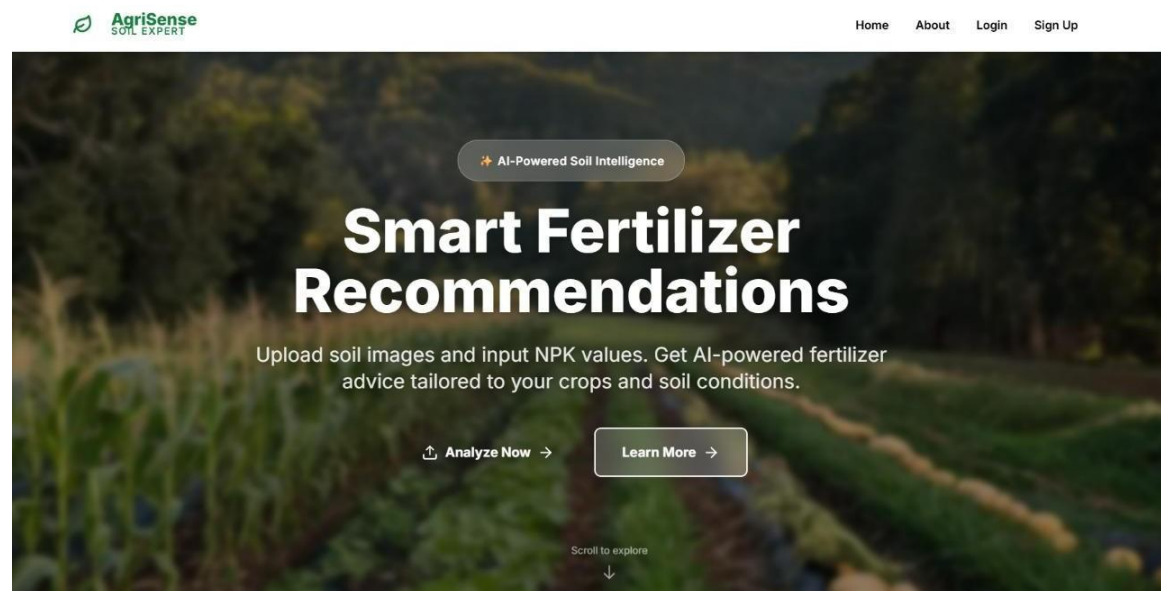


Figure 6.3: Home page

Figure 6.3 is the user homepage of web app provided to users. The home page features a visually appealing background of a green agricultural field that immediately establishes the platform's focus on farming and nature. A bold headline, "Smart Fertilizer Recommendations," sits at the center to highlight the system's core capability of using AI for soil intelligence. Users are guided by a concise description that explains how they can receive personalized advice by simply uploading soil images or entering specific NPK nutrient values. Two prominent buttons, "Analyze Now" and "Learn More," are strategically placed to encourage farmers to start the analysis process or explore the tool's features. Finally, a clean navigation bar at the top ensures easy access to account creation and login, while a subtle scroll indicator at the bottom invites users to discover more content.

6.4 Soil Analysis Page

Each input field provides instant visual feedback with status labels like "Moderate" or "Optimal," helping farmers immediately understand the condition of each nutrient. On the right side, a dynamic results panel displays a comprehensive "Soil Health Score," such as

the "100% Excellent" rating shown, to summarize the overall quality of the land. Finally, the system offers specific fertilizer recommendations based on the analysis, advising the user on whether to apply additives or simply maintain their current practices if conditions are optimal.

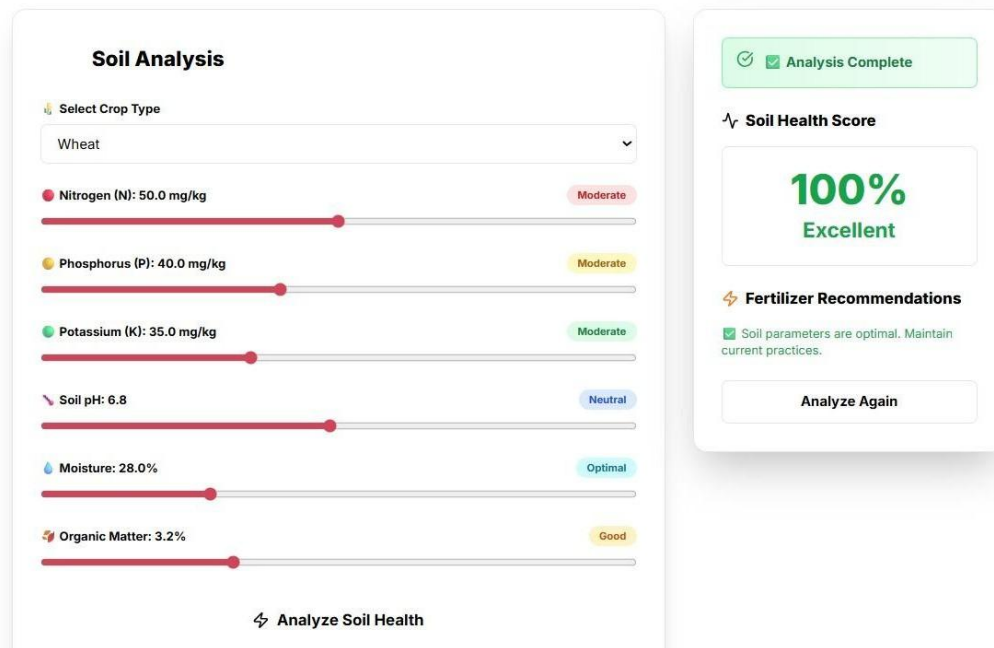


Figure 6.4: Soil Analysis Page

6.5 Plant Disease Detection Page

Figure 6.5 is the plant disease detection page of web app provided to users. The Disease Detection page is designed with a clear and intuitive interface that features a bold title and a microscope icon to immediately signal its diagnostic purpose. Users are guided by a helpful description that explains how uploading a simple photo can identify plant diseases early and provide necessary treatment recommendations. A prominent, central upload zone with a camera icon allows farmers to easily drag and drop images or click to select files from their device. To ensure compatibility and prevent errors, the tool explicitly states support for common image formats like JPG and PNG directly below the upload area. This streamlined design ensures that farmers can quickly submit their plant photos and receive instant health analysis without any technical complications.

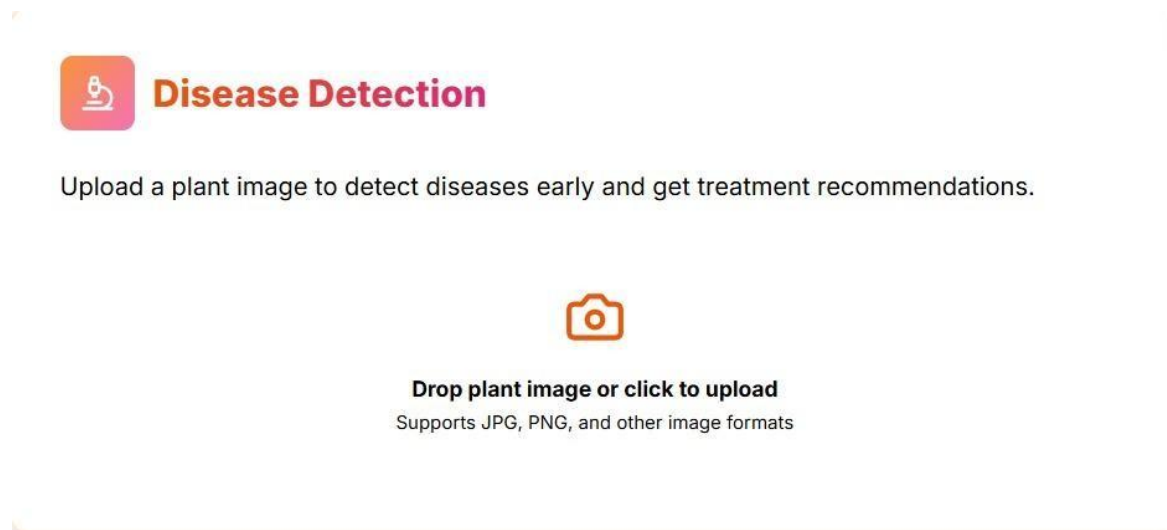


Figure 6.5: Plant Disease Detection Page

6.6 Process page



Figure 6.6: Process Page

Figure 6.6 is the process page of web app provided to users. The process page is titled "How AgriSense Works" and breaks down the complex task of agricultural analysis into three simple, user-friendly steps to guide farmers. First, users are instructed to input their specific soil parameters, such as nitrogen and pH levels, using easy-to-adjust sliders to capture accurate data. Next, the system prompts the farmer to select the specific crop they are cultivating so that the fertilizer recommendations are customized exactly for that plant's needs. The final step empowers users to monitor plant health by uploading images, allowing the system to detect potential diseases early and suggest effective treatments. This clean, sequential layout ensures that farmers can easily understand how to leverage the platform to get actionable insights for their land.

6.7 About Page

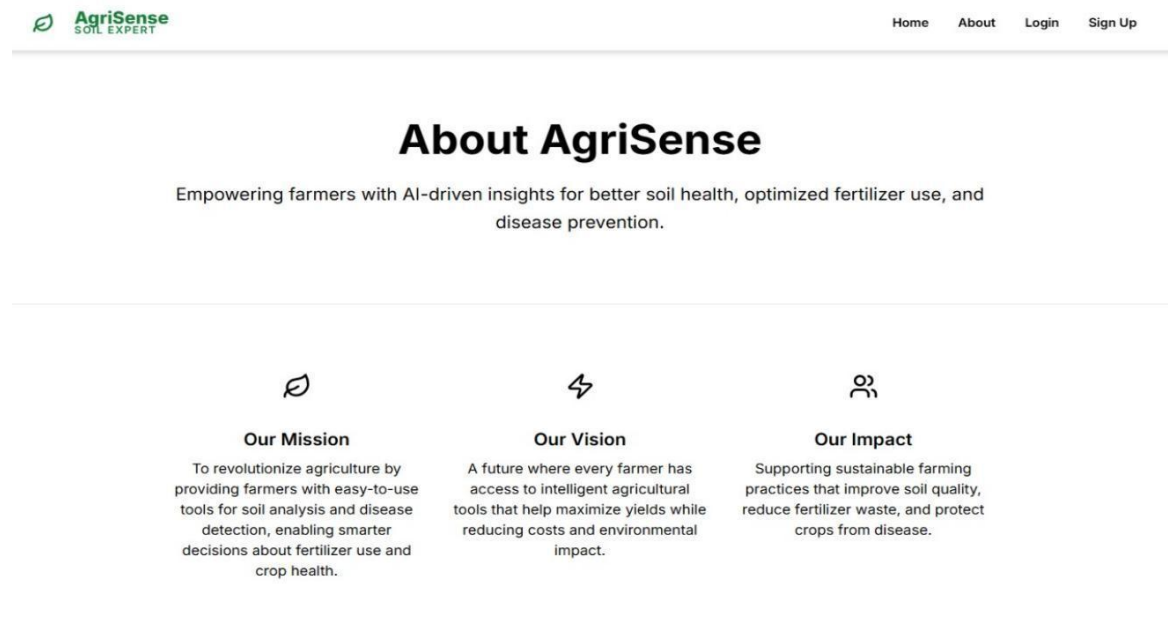


Figure 6.7: *About Page*

Figure 6.7 is the About page of web app provided to users. The "About AgriSense" page clearly defines the platform's goal of empowering farmers with AI-driven insights to improve soil health and prevent plant diseases effectively. The layout is organized into three distinct sections that outline the project's Mission, Vision, and intended Impact on the agricultural community. Under the Mission heading, the text highlights a commitment to revolutionizing agriculture by providing easy-to-use tools that help farmers make smarter decisions about fertilizer use and crop care. The Vision section outlines a future where intelligent agricultural technology helps every farmer maximize their crop yields while significantly reducing operational costs and environmental damage. Finally, the Impact section emphasizes the platform's dedication to sustainable farming practices that improve soil quality and reduce fertilizer waste to protect crops from disease.

CONCLUSION AND FUTURE SCOPE

This project successfully designed and developed a Plant-Specific Fertilizer, Soil, and Disease Recommendation System to provide an integrated, data-driven advisory tool for farmers. The system's implementation achieved its core goals by using a Random Forest model for accurate soil analysis and a Convolutional Neural Network (CNN) to identify plant diseases from images. These models were integrated via a FastAPI backend and presented through a React frontend, allowing the system to provide a holistic recommendation that considers both soil health and plant disease status simultaneously. The key contribution of this project is this holistic integration, which helps prevent the misuse of fertilizers and pesticides, thereby promoting sustainable farming practices, reducing operational costs for farmers, and contributing to improved crop yield.

Future enhancements will focus on integrating IoT sensors to enable real-time soil monitoring for dynamic and continuous recommendation updates. To ensure broader accessibility, the system will incorporate regional language support and voice-based interactions for users with limited technical literacy. A dedicated mobile application with offline capabilities is planned to guarantee functionality in remote areas with limited internet connectivity. Additionally, the models will be trained on a more diverse dataset of crop types and regional soil profiles to improve versatility and prediction accuracy. Finally, the platform aims to integrate a marketplace feature that connects farmers directly with suppliers for the recommended fertilizers and treatments.

REFERENCES

- [1] B. B. Tugrul, E. Elen, and R. Eryigit, "Convolutional neural networks in detection of plant leaf diseases: A review," *Agriculture*, vol. 12, no. 8, p. 1192, 2022.
- [2] P. P. R. et al., "Data-Driven Analysis and Machine Learning-Based Crop and Fertilizer Recommendation System for Revolutionizing Farming Practices," *Applied Sciences*, vol. 13, no. 11, p. 2141, Nov. 2023.
- [3] A. M. A. et al., "Integrating IoT for Soil Monitoring and Hybrid Machine Learning in Predicting Tomato Crop Disease in a Typical South India Station," *Sensors*, vol. 24, no. 19, p. 6177, Sep. 2024.
- [4] A. Rauf et al., "A Lightweight Deep Learning Model for Plant Disease Classification and Recommendation on Mobile Devices," *Applied Sciences*, vol. 13, no. 15, p. 8913, 2023.
- [5] S. Ramesh et al., "Soil Analysis, Disease Detection and Pesticide Recommendation for Farmers Using AI and ML," *Computer Science & Engineering: An International Journal (CSEIJ)*, vol. 15, no. 1, Feb. 2025.
- [6] V. S. et al., "Fertilizer Prediction Using Machine Learning," *International Journal for Research in Applied Science & Engineering Technology (IJRASET)*, vol. 11, no. 11, pp. 783-788, Nov. 2023.
- [7] A. Divya, R. Josphineleela, and L. J. Sheela, "A machine learning based approach for prediction and interpretation of soil properties from soil spectral data," *Journal of Environmental Biology*, vol. 45, pp. 96-105, 2024.
- [8] M. S. et al., "IOT-Based Soil Monitoring and Integrated Pest and Disease Detection System," *International Journal of Creative Research Thoughts (IJCRT)*, vol. 13, no. 5, May 2025.
- [9] S. T. S. B., S. N. A. S., and M. A., "A review on: Deep learning in plant disease detection," *Journal of Physics: Conference Series*, vol. 1892, no. 1, p. 012015, 2021.
- [10] A. Ahmad, S. Al-Shara, M. F. Al-Bodour, and O. M. Al-Kj., "An Efficient and Lightweight CNN Model for Plant Disease Detection," *Electronics*, vol. 11, no. 3, p. 372, Jan. 2022.
- [11] S. L. A, N. M. B, J. M. A, and S. A. A, "Plant Disease Detection Using Deep Learning: A Review," *Electronics*, vol. 12, no. 12, p. 2736, 2023.
- [12] J. K. K, P. K, and S. V, "Machine Learning-Based Crop Recommendation System for Smallholder Farmers in India," *AgriEngineering*, vol. 5, no. 1, pp. 294-308, 2023.